Progress Evaluation

Final Year Project

FYP

February 2026

Contents

1 Introduction

1.1 Background

1.2 Motivation

1.3 Research Objectives

1.4 Novel Research Contribution

2 Literature Review

2.1 Mixture of Experts architectures (FFN, LoRA, distilled LLM experts)

2.2 Routing strategies (top-k, adaptive, expert-choice, RL-based)

2.3 Multilingual LLMs and cross-lingual challenges

2.4 Parameter-efficient fine-tuning (LoRA, QLoRA)

2.5 Research gap addressed by this project

3 Methodology

3.1 Overall system architecture (Three-stage routing + expert pool)

The proposed system follows a modular Mixture of LLM-based Experts (MoE) architecture combined with a multi-stage routing pipeline to efficiently handle multilingual and multi-task text classification. The core design principle is to progressively narrow down candidate experts through lightweight routing stages before activating a single specialized expert from a shared expert pool. This ensures scalability, computational efficiency, and strong specialization while avoiding unnecessary expert activation.

At a high level, the architecture consists of three main components: (i) a multi-stage routing pipeline, (ii) an expert pool composed of specialized LLM experts, and (iii) an orchestration layer that coordinates routing decisions and expert invocation.

The three-stage routing pipeline is designed to decompose the expert selection problem into simpler, interpretable steps:

Stage 1 – Language Routing: The input text is first processed by a lightweight language identification module. This stage determines the input language and restricts downstream processing to experts trained or adapted for that specific language. By filtering experts early, the system reduces routing complexity and mitigates cross-lingual interference, especially for low-resource languages.

Stage 2 – Task Identification: After language filtering, the input is passed to a task classification module that identifies the target task (e.g., sentiment analysis, topic classification, product relevance classification). This stage further narrows the candidate

expert set by selecting only those experts fine-tuned for the identified task within the detected language group.

Stage 3 – Expert Selection: In the final stage, the system selects the most suitable expert from the reduced candidate set. This selection can be performed using either a static routing strategy (rule-based or classifier-based mapping) or a dynamic routing strategy optimized through reinforcement learning. The output of this stage is a single expert model that processes the input and produces the final classification result.

The expert pool consists of multiple LLM-based experts, each specialized along dimensions such as language, task, or domain. Instead of maintaining entirely separate models, experts are implemented using parameter-efficient fine-tuning techniques (e.g., LoRA or QLoRA) applied on top of shared base LLMs. This design significantly reduces memory and training costs while preserving expert specialization. Experts are registered in a centralized registry that maintains metadata such as supported languages, tasks, and fine-tuning configurations, enabling efficient lookup during routing.

Overall, this three-stage routing plus expert pool architecture enables conditional computation, where only a single specialized expert is activated per input. Compared to dense multilingual models, the architecture improves efficiency and robustness by avoiding unnecessary parameter activation. At the same time, its modular structure allows new experts, languages, or tasks to be added without retraining the entire system, making it well-suited for evolving multilingual text classification scenarios.

3.2 Stage 1: Language detection (FastText)

Language detection is a key early step in the proposed Mixture of LLM-based Experts framework for multilingual text classification. Before routing an input to a suitable expert model, the system must first identify the language of the given text in a fast and reliable manner. Accurate language detection helps reduce unnecessary computation and ensures that only experts trained for the detected language are considered in later stages of the pipeline.

In this project, language detection is implemented as a lightweight and independent module that operates before task and expert selection. The module uses a pretrained FastText language identification model that supports a large number of languages, allowing broad multilingual coverage. To improve robustness, especially for short, noisy, or low-quality inputs, simple rule-based fallback methods based on character patterns and common words are also included. This hybrid approach ensures stable language detection even when model confidence is low.

The detected language is used as the first decision point in the hierarchical routing strategy described in the proposed framework, where inputs are first grouped by language and then routed to task-specific experts. By filtering experts at the language level, the system improves classification accuracy and reduces routing complexity. This design supports scalability, as new languages can be added by extending the language detection module without changing the overall architecture, making it well suited for multilingual and multi-domain text classification systems

3.3 Stage 2: Task Identification

After identifying the input language in Stage 1, the second stage of the routing pipeline performs task identification. Since this project focuses on finance-oriented multilingual datasets, all inputs are assumed to belong to a single domain. Therefore, the routing objective at this stage is to determine the correct task category associated with the input.

Task identification is an essential step because the supported tasks differ significantly in terms of input structure, label space, and expected output format. For example, sentiment rating prediction requires classification into a discrete 1–5 scale, while query–product relevance classification requires relational reasoning between two text segments. Similarly, PII extraction requires generating structured entity-level outputs rather than producing a single label. As a result, incorrect task identification can lead to inappropriate expert activation and invalid predictions.

To ensure efficiency, task identification is implemented as a lightweight neural classifier that assigns a discrete task label to each input prompt. This task label serves as an intermediate routing signal that guides downstream expert selection, allowing the routing system to make task-aware decisions while remaining scalable as new tasks are introduced.

Overall, Stage 2 enables multi-task routing by providing an explicit task-level decomposition of the input space, improving both interpretability and robustness of the overall MoE framework.

3.4 Stage 3: Expert Selection

3.4.1 Static

3.4.2 Dynamic

3.5 Expert design (registry, language-to-expert mapping, adapter pool)

3.6 Evaluation Matrix

The proposed Mixture of LLM-based Experts (MoE) framework is evaluated using a comprehensive set of metrics designed to assess classification correctness, label-level consistency, and comparative model behavior across languages and tasks. These metrics are aligned with the experimental results obtained during baseline evaluations and expert benchmarking, ensuring that all reported measures are empirically grounded and reproducible.

. **Accuracy:** Accuracy serves as the primary metric for measuring the proportion of correctly classified instances. It offers a clear and interpretable indicator of overall task performance, making it particularly suitable for comparing different model architectures and routing strategies under consistent dataset and label configurations. In baseline comparisons, such as the MultiFin experiments, accuracy is used to establish a global performance ranking among candidate models and experts.

2. Precision, Recall, and Macro F1-Score: To provide a more granular assessment beyond overall accuracy, precision, recall, and macro-averaged F1-score are employed. Precision quantifies the correctness of predicted labels, recall captures the model's capacity to identify all relevant labels, and the macro F1-score computes the harmonic mean of precision and recall independently for each class before averaging across all classes. Among these, the macro F1-score is emphasized as the principal evaluation metric, as it offers a balanced view of performance across all classes and mitigates the risk of dominant classes masking weaker predictions.

3. Token-Level Accuracy: In addition to instance-level correctness, token-level accuracy evaluates how accurately models generate or align token sequences with the expected output. This metric captures partial correctness in structured or multi-token outputs, providing finer-grained insight than exact-match accuracy alone.

4. Exact Match Accuracy: Exact match accuracy measures the percentage of predictions that precisely replicate the ground-truth label sequence. Although strict in nature, this metric serves as a strong indicator of end-to-end correctness and is valuable for identifying performance gaps between models that exhibit similar token-level behavior.

5. Task-Wise and Language-Wise Aggregation: All classification metrics are systematically aggregated along two dimensions: per task, to assess task-specific expert effectiveness, and per language, to compare relative model performance and inform expert selection.

6. Representation Similarity Analysis: To examine cross-language prediction behavior, cosine similarity is computed between model output representations relative to a designated reference language. This metric provides insight into the degree of alignment between predictions across different languages in the embedding space.

7. Baseline Comparison Strategy: The aforementioned metrics are collectively applied to compare dense multilingual baseline models, candidate expert models prior to fine-tuning, and static expert routing configurations.

Overall, the evaluation framework is centered on classification accuracy, label-level robustness, and comparative consistency objectives that directly support expert selection and routing within the proposed MoE framework.

4 Datasets and Tasks

The proposed Mixture of LLM-based Experts (MoE) framework is evaluated using four multilingual datasets drawn from established benchmarks in financial and e-commerce natural language processing. These datasets are selected to cover a range of task formulations, including classification and structured extraction, while maintaining a consistent application domain. Collectively, the experimental setup spans four distinct tasks and 12 languages, providing a diverse and realistic setting for evaluating expert specialization and routing strategies.

4.1 Task Overview

The proposed MoE framework is evaluated across four complementary financial NLP tasks, each selected to stress a different aspect of multilingual and multi-task modeling. Together, these tasks span both classification and structured extraction, enabling a comprehensive assessment of expert specialization and routing behavior.

- **Sentiment Rating Prediction** focuses on inferring a discrete 1–5 star rating from multilingual Amazon product reviews, requiring robust semantic understanding of user opinions across languages.
- **Financial News Classification** involves assigning finance-related news headlines to predefined topical categories, testing the system's ability to capture domain-specific cues within short textual inputs.
- **Query–Product Relevance Classification** evaluates relational reasoning by identifying the semantic relationship between a user search query and a candidate product description.
- **PII Entity Extraction** addresses structured information extraction from financial documents, requiring precise detection and classification of personally identifiable information.

These tasks differ substantially in input structure, label space, and output representation, ranging from single-label classification to structured JSON outputs. This diversity allows the MoE framework to be evaluated under heterogeneous conditions while maintaining a unified routing and execution pipeline.

4.2 Dataset Descriptions

4.2.1 MARC — Multilingual Amazon Reviews Corpus

The Multilingual Amazon Reviews Corpus (MARC) is a large-scale benchmark dataset for multilingual sentiment analysis, introduced by Keung et al. (2020) at EMNLP 2020. MARC contains Amazon product reviews across six languages: English, German, Spanish, French, Japanese, and Chinese, collected between November 2015 and November 2019. Each review is annotated with a 1–5 star rating.

The dataset includes review text, review title, star rating, anonymized reviewer and product IDs, and product category metadata. A key feature is its balanced class distribution: each star rating constitutes exactly 20% of reviews in each language. The original dataset provides 200,000 training, 5,000 development, and 5,000 test samples per language.

In this work, a curated subset of MARC is used covering all six languages. Each instance combines the review title and body into a single textual input, with the product category appended where available. The prepared dataset consists of 8,160 training samples and 1,020 test samples, evenly distributed across languages.

4.2.2 MultiFin — Multilingual Financial News Dataset

MultiFin is a multilingual financial news classification dataset developed by Jørgensen et al. (2023) at EACL 2023. The dataset consists of 10,048 finance-related news head-

lines across 15 languages: English, Spanish, Polish, Hungarian, Greek, Danish, Turkish, Japanese, Swedish, Finnish, Norwegian, Russian, Italian, Hebrew, and Icelandic. This represents diverse language families including Germanic, Romance, Slavic, Turkic, Finno-Ugric, and Hellenic languages.

The dataset features a hierarchical label structure with 6 high-level topic categories (Finance, Tax & Accounting, Government & Controls, Technology, Industry, Business & Management) and 23 low-level labels. The original dataset is partitioned into 6,430 training, 1,608 development, and 2,010 test samples. Annotations were developed using both 'label by native-speaker' and 'translate-then-label' approaches.

For this work, the task uses the six-class high-level taxonomy. A subset of 5 languages (Danish, English, Spanish, Polish, Turkish) is selected, with 3,200 training samples and 400 test samples evenly distributed across languages.

4.2.3 Amazon ESCI — Query–Product Relevance Dataset

The Amazon ESCI (Exact, Substitute, Complement, Irrelevant) dataset, also known as the Shopping Queries Dataset, was released by Amazon Science to advance research in e-commerce search and semantic matching of queries and products. For each query, the dataset provides up to 40 potentially relevant product results with detailed ESCI relevance judgements.

Query–product pairs are annotated according to four semantic relationship categories: (i) Exact (E) — direct match, (ii) Substitute (S) — serves similar purpose, (iii) Complement (C) — related/often purchased together, and (iv) Irrelevant (I) — no meaningful relationship. The class distribution is: 65.17% Exacts, 21.91% Substitutes, 2.89% Complements, and 10.04% Irrelevants. Annotations are derived from both user behavior data and explicit human judgments.

The dataset is multilingual, containing queries and products in English, Japanese, and Spanish. It references approximately 1.8 million unique products and includes comprehensive product metadata (title, description, bullet points, brand, color, etc.). The dataset was used for the KDD Cup 2022 ESCI Challenge, which drew over 1,600 participants and 9,200 submissions.

In this work, the task is to predict one of the four relevance labels for each query–product pair. The test set includes 459 samples across English, Spanish, and Japanese.

4.2.4 Gretel AI Finance PII Synthetic Dataset

The Gretel AI Finance PII Synthetic Dataset (gretelai/synthetic_pii_finance_multilingual) is a privacy-preserving benchmark for evaluating PII extraction models in financial documents. Released by Gretel AI in June 2024 under Apache 2.0 license, the dataset is entirely synthetically generated using Gretel Navigator, ensuring it can be freely used without privacy concerns.

The complete dataset contains 55,940 records (50,776 training, 5,164 test) with coverage across 100 distinct financial document formats including transaction statements, credit card records, loan applications, and SWIFT messages. Documents average 1,357

characters in length. The dataset spans seven European languages: Dutch, English, Spanish, French, German, Italian, and Swedish.

Each document is annotated with 29 distinct PII types aligned with Python Faker library generators, including person names, email addresses, phone numbers, physical addresses, dates of birth, account numbers, credit card numbers, social security numbers, tax IDs, IP addresses, organization names, and various other financial identifiers. Annotations include exact text spans (start/end positions), entity types, and positional indices.

The dataset generation involved multiple quality assurance steps including LLM-based document generation, PII span labeling, validation using the Gliner NER library, human review, and LLM-as-a-Judge evaluation on conformance, quality, toxicity, and bias scores.

In this work, the PII extraction task requires models to identify entities and output structured annotations containing entity text, entity type, and occurrence index. The prepared dataset includes 8,348 training samples and 1,047 test samples.

5 Progress

5.1 Input Data Processing

All datasets are processed through a unified input preparation pipeline to ensure consistency across tasks and languages. This pipeline converts heterogeneous raw sources into a standard JSON representation suitable for training, evaluation, and routing within the MoE architecture.

5.1.1 Data Cleaning and Normalization

Raw dataset files are read from CSV sources and cleaned to remove incomplete or malformed entries. Language identifiers originating from different conventions (e.g., full language names or variant codes) are normalised into consistent two-letter ISO-style codes using a dedicated mapping layer. Task-specific fields are extracted according to the dataset type. For example, review titles and bodies are concatenated for sentiment analysis, query and product description fields are separated for ESCI, and synthetic document text is extracted for PII processing.

MARC Dataset Processing: For the Multilingual Amazon Reviews Corpus, review title and review body fields are concatenated into a single input text. Product category information is appended where available to provide additional context. The 1–5 star ratings are preserved as the ground truth labels. Each sample is assigned a language code based on the source marketplace.

MultiFin Dataset Processing: Financial news headlines are extracted as the primary input text. The hierarchical label structure of the original dataset (6 high-level and 23 low-level categories) is simplified to use only the 6 high-level topic categories: Finance, Tax & Accounting, Government & Controls, Technology, Industry, and Business & Management. Language codes are normalized across the 15-language dataset, with a subset of 5 languages (Danish, English, Spanish, Polish, Turkish) selected for the experimental evaluation.

Amazon ESCI Dataset Processing: Query and product description fields are extracted separately and maintained as paired inputs. Product metadata including product title, product description, and product bullet points are concatenated to form the complete product representation. The four-class ESCI labels (Exact, Substitute, Complement, Irrelevant) are preserved without modification. Language-specific samples are filtered based on the product locale field to create language-stratified evaluation sets.

Gretel AI PII Dataset Processing: The synthetic financial documents undergo substantial preprocessing to convert the original 29 distinct PII entity types into a simplified classification schema suitable for extraction evaluation. The original PII types, which are aligned with Python Faker library generators, are mapped to 11 higher-level categories as follows:

- **person_name**: first_name, last_name, name
- **date**: date, date_of_birth, date_time, time
- **location**: street_address, local_latlng
- **organization**: company
- **contact_info**: email, phone_number
- **government_id**: ssn, passport_number, driver_license_number
- **financial_account**: bank_routing_number, bban, iban, swift_bic_code, account_pin
- **payment_card**: credit_card_number, credit_card_security_code
- **user_identifier**: customer_id, employee_id, user_name
- **secret**: password, api_key
- **ip_address**: ipv4, ipv6

The original span-based annotation format (start position, end position, entity type) is transformed into an evaluation-friendly structure through the following steps: (i) text is extracted from the provided character span ranges in each document, (ii) each extracted entity is assigned its corresponding simplified PII label based on the mapping above, (iii) an occurrence index is added to distinguish repeated entities of the same type within a single document (e.g., if a document contains three instances of 'person_name', they are labeled as occurrence 1, 2, and 3 respectively), and (iv) a final annotation structure is created containing the extracted entity text, simplified PII label, and occurrence index. This restructured format replaces the original start/end span system with a representation that is more suitable for evaluating instruction-tuned language models, which are expected to generate structured output rather than perform sequence labeling.

5.1.2 Prompt Construction

For each task and language, predefined natural language prompt templates are applied to construct the final model input. Templates are stored externally and selected randomly per instance using a fixed random seed (42) to ensure reproducibility. Where language-specific templates are unavailable, English templates are used as a fallback. Ground-truth labels are never included in the prompt text, ensuring a clean separation between input and supervision.

5.1.3 Unified JSON Representation

After preprocessing, all samples are converted into a unified JSON schema containing: the generated prompt, the raw input content, the normalised language code, the task identifier, the domain label (finance), and the ground-truth annotation. This unified representation allows the routing system to operate independently of dataset-specific formats.

5.1.4 Record Ordering for Efficient Evaluation

To improve evaluation efficiency, processed records are ordered by task and language prior to inference. This ordering reduces repeated expert or adapter switching during batched evaluation, as consecutive samples often share the same task-language configuration.

5.2 Model Selection and Fine-tuning

5.3 Language Detector Implementation

Language detection is implemented as a lightweight, standalone component that operates at the very beginning of the proposed Mixture of LLM-based Experts framework. Its primary responsibility is to identify the language of an input text and provide this information to the hierarchical routing mechanism. Since expert selection in the framework is language-aware, incorrect language identification can lead to routing the input to unsuitable experts, reducing both accuracy and efficiency.

To address this, the language detection module is designed to be fast, reliable, and independent from the heavier LLM-based components used later in the pipeline. This ensures that language identification does not become a bottleneck in the overall system.

5.3.1 Core Model and Initialization

The main language detection capability is provided by a pretrained FastText language identification model (lid.176.bin), which supports 176 languages. During system initialization, the model is loaded once and reused for all subsequent inputs. If the model file is not found locally, it is automatically downloaded from the official FastText repository and stored for future use. The module maintains a mapping between FastText language labels (e.g., $_{label_{en)and human-readable language names (e.g., english). This mapping allows the rest of the system to work with consistent language identifiers, regar}}$

5.3.2 Dynamic Language Support Configuration

To support the Mixture of Experts framework, the language detector can optionally load a registry configuration that specifies which languages are supported for each task. During initialization, the module reads this registry and:

extracts the language codes associated with each task,

converts them into full language names, and

builds a dynamic label-to-language mapping based only on the languages actually supported by the system.

If no registry is provided, the detector falls back to a comprehensive predefined language mapping. This design allows the system to adapt easily when new languages or tasks are added, without requiring changes to the core detection logic

5.3.3 Language Detection Process at Runtime

When a new input text is received, the following steps are performed:

Text Preprocessing The input text is cleaned by removing line breaks and trimming whitespace. Very short inputs (e.g., fewer than three characters) are handled with a default fallback to avoid unreliable predictions.

FastText Prediction The cleaned text is passed to the FastText model, which predicts the most likely language label along with a confidence score. Only the top prediction is used, since language detection is treated as a single-label classification problem.

Label Mapping The predicted FastText label is mapped to a full language name using the dynamically constructed mapping. If the label is not found, the system defaults to English to maintain safe behavior.

Error Handling If any error occurs during prediction (e.g., model failure or unexpected input), the system automatically switches to a fallback detection mechanism.

5.3.4 Fallback Detection Mechanism

The fallback detection logic is implemented to handle cases where:

the FastText model is unavailable,

the input text is too short, or

the prediction confidence is unreliable.

This fallback approach combines two simple but effective techniques:

Script-Based Detection Unicode character ranges are used to identify languages with distinct writing systems, such as Chinese, Japanese, Korean, Arabic, Hindi, and Russian. The presence of characters from these ranges strongly indicates the corresponding language.

Keyword-Based Detection For alphabet-based languages, the system checks for common words (such as articles, conjunctions, and frequently used terms). Each detected match contributes to a score for that language, and the language with the highest score is selected.

5.4 Task Classifier Implementation

5.5 Expert Selector Implementation

5.5.1 Static

5.5.2 Dynamic

We experiment and implement a dynamic routing mechanism that selects the most suitable Large Language Model (LLM) to classify a given input text into the correct class or label.

Motivation for a Dynamic Router

A single multilingual LLM often fails to perform consistently across multiple languages and domains. Even within the same task, different LLMs may exhibit specialized strengths in certain languages or sub-domains.

For example, in the Financial News Headlines Classification task, class labels include Business & Management, Finance, Government & Controls, Technology, Industry, and Tax & Accounting. An LLM such as Aya-23 may correctly classify Japanese finance-related headlines, while performing poorly on Japanese Business & Management headlines.

Due to such performance variability, relying on a single LLM can limit overall accuracy. To address this issue, we propose a dynamic router that learns to select the most suitable LLM for a given input based on its characteristics.

To model this decision-making process, we explore reinforcement learning based approaches, and in this work we focus on a neural contextual bandit formulation.

Elements of the Router

1. Input Embeddings: We use XLM-RoBERTa (XLM-R) to generate contextual embeddings for the input text. XLM-R is selected due to its strong multilingual capabilities, having been trained on data from over 100 languages. This makes it well suited for our multilingual classification scenario.

2. Pooling Layer: The pooling layer aggregates token-level embeddings into a single sentence-level representation. In the current system, we employ mean pooling, where token embeddings are averaged while excluding padding tokens using the attention mask.

Mean pooling provides a simple and stable sentence representation. In future work, we plan to experiment with task-aware pooling strategies to further improve routing performance.

3. Deep Neural Network: A deep neural network is used as a function approximator to estimate the expected reward (Q-value) for each LLM in the model pool, given the input context. The output of this network is a vector of Q-values, one per LLM, representing the estimated suitability of each model for the given input.

Methodology

When an input text reaches the dynamic routing component, the following steps are performed.

1. Embedding Generation: The input text is first tokenized and encoded using XLM-R, producing token-level contextual embeddings.

2. Sentence Representation: Mean pooling is applied to obtain a fixed-dimensional sentence embedding.

3. Language Encoding: A one-hot language vector corresponding to the detected language of the input text is generated.

4. Context Construction: The final context vector is formed by concatenating the sentence embedding with the language vector:

Context = XLM-R sentence embedding + language embedding

5. LLM Selection: The context vector is passed to the router network, which estimates Q-values for each LLM. Based on these values, the router selects the most suitable LLM to classify the input text.

6. Prediction: The selected LLM produces the final classification output.

Model Training

The routing task is formulated as a contextual bandit problem. At each decision step:

1. The router observes a context vector derived from the input text.
2. It selects one LLM from the available pool.
3. Only the reward of the selected LLM is observed.

Rewards for unselected models are not available, reflecting realistic routing constraints.

Action Space

The action space consists of selecting one LLM from the predefined model pool. For the current experiments we chose three multilingual LLMs.

$$\mathcal{M} = \{Gemma, Aya, BloomZ\}$$

Reward Function

After the selected LLM produces a prediction, a binary reward is computed based on classification correctness.

Exploration Strategy

To balance exploration and exploitation, the router uses an ε-greedy policy during training.

With probability $1-\varepsilon$, the router selects the LLM with the highest estimated Q-value.

Experience Replay

Each interaction is stored in a replay buffer as a tuple:

$$\mathcal{S} = (Context, Action, Reward)$$

Mini-batches are sampled uniformly from the replay buffer to train the router. Experience replay improves data efficiency and stabilizes learning by reducing correlations between consecutive updates.

5.6 Baseline Comparison

6 References